

Прийоми ефективного програмування

Частина 1. Ітератори та генератори

Ітератори та генератори: загальні поняття

Один з прийомів ефективного програмування мовою Python передбачає використання ітераторів та генераторів. З точки зору функціональності ітератор і генератор — майже одне й те саме (ітератор — трохи ширше поняття, бо кожен генератор є ітератором). Основна відмінність полягає в програмній реалізації, а саме: генератори реалізуються у вигляді функцій, а ітератори — класів.

Ми вже використовували вбудовані генератори для генерування колекцій: списків, словників, множин. Наприклад, вираз

```
[x**2 for x in range(10**6)]
```

генерує список квадратів перших мільйон натуральних чисел (починаючи з 0).

Основним призначенням ітераторів і генераторів є економія пам'яті. Припустимо, нам потрібно порахувати суму цих чисел. Для цього можна застосувати функцію `sum`. Але, попри те, що нам потрібне лише значення суми, а не самі ці числа, в оперативній пам'яті зберігатиметься список усього мільйона чисел. Обсяг займаної пам'яті можна обчислити так:

```
from sys import getsizeof

lst = [x**2 for x in range(10**6)]
print('Sum: ', sum(lst))
print('Memory:', getsizeof(lst))
```

Результатом виконання коду буде:

```
Sum:  333332833333500000
Memory: 8697456
```

Тобто, об'єкт списку `lst` займає приблизно 8,7 Мб.

У наведеному вище коді в квадратних дужках знаходиться об'єкт генератора, який генерує числа від 0 до 999999 а самі дужки виконують функцію створення списку з цих чисел. Якщо цей об'єкт записати в окрему

змінну, то ми теж отримаємо значення суми, але не зберігатимемо в пам'яті усі числа. Продемонструємо це:

```
from sys import getsizeof

gen = (x**2 for x in range(10**6))
print('Sum: ', sum(gen))
print('Memory:', getsizeof(gen))

# результат виконання коду:
Sum:  333332833333500000
Memory: 112
```

Як бачимо, обсяг пам'яті зменшився більше ніж у 77 тисяч разів. Це тому, що генератор по генерував числа по одному і передавав їх у функцію `sum`, а не створював увесь список з мільйона чисел. Різницю можна помітити, роздрукувавши об'єкти `lst` і `gen`:

```
print('lst: ', lst)
print('gen: ', gen)

# результат виконання коду:
lst: [0, 1, 2, ..., 999998, 999999]
gen:  <generator object <genexpr> at 0x7fe1ad4bfff90>
```

Отже, образно кажучи, генератор та ітератор — це такі об'єкти, які під час створення не обчислюють одразу значення всіх своїх елементів. Вони зберігають в пам'яті тільки останній обчислений елемент, правило переходу до наступного елемента і умову завершення. Обчислення наступного значення здійснюється тільки під час виклику методу `__next__()`, при цьому попереднє значення втрачається.

Функція/метод `next()`

Продемонструємо роботу методу `next()` на прикладі циклів `while` і `for`.

```
x = (x**2 for x in range(4))
while True:
    print(next(x))

# результат виконання коду:
0
```

```

1
4
9
Traceback (most recent call last):
  File "/usr/lib/python3.8/idlelib/run.py", line 559, in runcode
    exec(code, self.locals)
  File "/home/mokasin/w.py", line 6, in <module>
    print(next(x))
StopIteration
>>>

```

Як бачимо, метод `next()` генератора, який викликається однойменною функцією, в кожній ітерації циклу по черзі видає квадрати чисел з діапазону `range(4)`, тобто, квадрати чисел 0, 1, 2 і 3. Наступний після цього виклик методу викликає специфічну помилку `StopIteration`, оскільки генератору більше немає що генерувати. Для того, щоб програма завершувалася коректно, потрібно задіяти механізм обробки винятків:

```

x = (x**2 for x in range(4))
while True:
    try:
        print(next(x))
    except:
        break

# результат виконання коду:
0
1
4
9

```

Внутрішня реалізація циклу `for` передбачає автоматичний виклик `next()` та обробку винятку `StopIteration`, тому програма буде коротшою і завершиться коректно без інструкції `try`:

```

x = (x**2 for x in range(4))
for item in x:
    print(item)

# результат виконання коду:
0
1
4
9

```

Функцію можна запускати не тільки в одному циклі, а й окремо, чи навіть в різних циклах. Спробуйте самостійно пояснити результат виконання такої програми:

```
x = (x**2 for x in range(10))

print('print 1: ', next(x))

for i in range(2):
    print('print 2: ', next(x))

print('print 3: ', next(x))
print('print 4: ', next(x))

while True:
    try:
        print('print 5: ', next(x))
    except:
        break

# результат виконання коду:
print 1:  0
print 2:  1
print 2:  4
print 3:  9
print 4:  16
print 5:  25
print 5:  36
print 5:  49
print 5:  64
print 5:  81
```

Функція/метод iter()

Функція iter() дає змогу створювати об'єкт ітератора, який по чергово повертає елементи переданої їй колекції:

```
collection = [0, '1', 'two', 3]
x = iter(collection)
print(x)

while True:
    try:
```

```
        print(next(x))
    except:
        break
```

```
# результат виконання коду:
<list_iterator object at 0x7f5c8c947f40>
0
1
two
3
```

Функції `iter()` можна передавати викликний (Callable) об'єкт. При цьому вона повинна приймати другий аргумент - генерування значення цього аргумента спричинить помилку `StopIteration`. Переконайтеся у цьому на такому прикладі:

```
from random import randint

def dice():
    return randint(1, 6)

dice_iter = iter(dice, 1)

for i in dice_iter:
    print(i)
```

Навчимося тепер створювати власні (користувацькі) ітератори та генератори.

Ітератори

Продемонструємо процес реалізації ітератора на прикладі створення ітератора, який імітує підкидання грального кубика задану кількість разів. Код цього ітератора має вигляд:

```
class DiceImitator:

    def __init__(self, limit):
        from random import randint
        self.randint = randint
        self.limit = limit
        self.counter = 0
```

```

def __next__(self):
    if self.counter < self.limit:
        self.counter += 1
        return self.randint(1, 6)
    raise StopIteration

def __iter__(self):
    return self

```

Як було зазначено вище, ітератор реалізується у вигляді класу. У конструкторі ініціалізуються основні атрибути-властивості ітератора, у нашому випадку це:

- імпортована з модуля `random` функція `randint()`, яка повертає випадкове ціле число з переданого їй діапазону і потрібна для імітації підкидання грального кубика;
- загальна кількість підкидань, яка передається конструктору під час створення класу;
- лічильник підкидань, який ініціалізується початковим значенням 0.

Метод `__next__()` є саме тим методом, який робить наш клас `DiceImitator` ітератором. Цей метод викликається під час запуску функції `next()`. У ньому ми перевіряємо, чи не закінчилися ще підкидання. Якщо ні, то повертаємо випадкове ціле число від 1 до 6 і збільшуємо лічильник підкидань на 1, якщо так — викликаємо виняток `StopIteration`, який сигналізує про завершення роботи ітератора.

Стандартний метод `__iter__()`, який просто повертає `self`, необхідний для коректної роботи ітератора з циклом `for`. Без цього методу спроба використати ітератор у циклі `for` завершиться помилкою `TypeError: 'DiceImitator' object is not iterable`. На цикл `while` наявність чи відсутність цього методу ніяк не впливає.

Продемонструємо роботу нашого ітератора:

```

game1 = DiceImitator(10)
game2 = DiceImitator(3)

print('\nGame 1')
for i in game1:
    print(i)
print('Game over')

print('\nGame 2')

```

```
while True:
    try:
        print(next(game2))
    except:
        print('Game over')
        break
```

В результаті отримаємо дві імітації низки підкидань грального кубика, з 10-и та 3-х підкидань.

```
Game 1
2
6
5
1
4
2
1
2
5
2
Game over
```

```
Game 2
6
1
3
Game over
```

Генератори

Реалізуємо тепер імітацію підкидань грального кубика у вигляді генератора. Як було зазначено вище, генератор реалізується у вигляді функції. Особливість, яка робить функцію генератором, полягає у використанні інструкції `yield` замість інструкції `return`. Виконання інструкції `yield` не завершує виконання функції, а лише призупиняє її до наступного виклику.

```
def dice_imitator(limit):
    from random import randint
    for _ in range(limit):
        yield randint(1, 6)

game1 = dice_imitator(10)
game2 = dice_imitator(3)
```

```
print('\nGame 1')
for i in game1:
    print(i)
print('Game over')

print('\nGame 2')
while True:
    try:
        print(next(game2))
    except:
        print('Game over')
        break
```

Результат виконання буде таким самим, як і у випадку ітератора (звісно, будуть згенеровані інші випадкові числа).

Модуль `itertools`

Стандартна бібліотека Python містить модуль `itertools` з уже реалізованими ітераторами, готовими для використання. Наведений нижче код демонструє використання функцій цього модуля для генерування перестановок та комбінацій елементів списку:

```
from itertools import permutations, combinations

names = ['Alice', 'Bob', 'Kate', 'Peter']

print("\nПерестановки імен:")
for item in permutations(names):
    print(f'\t{item}')

print("\nКомбінації з двох імен")
for item in combinations(names, 2):
    print(f'\t{item}')
```

Результат виконання цього коду наступний:

```
Перестановки імен:
('Alice', 'Bob', 'Kate', 'Peter')
('Alice', 'Bob', 'Peter', 'Kate')
('Alice', 'Kate', 'Bob', 'Peter')
('Alice', 'Kate', 'Peter', 'Bob')
('Alice', 'Peter', 'Bob', 'Kate')
```



```
('Alice', 'Peter', 'Kate', 'Bob')
('Bob', 'Alice', 'Kate', 'Peter')
('Bob', 'Alice', 'Peter', 'Kate')
('Bob', 'Kate', 'Alice', 'Peter')
('Bob', 'Kate', 'Peter', 'Alice')
('Bob', 'Peter', 'Alice', 'Kate')
('Bob', 'Peter', 'Kate', 'Alice')
('Kate', 'Alice', 'Bob', 'Peter')
('Kate', 'Alice', 'Peter', 'Bob')
('Kate', 'Bob', 'Alice', 'Peter')
('Kate', 'Bob', 'Peter', 'Alice')
('Kate', 'Peter', 'Alice', 'Bob')
('Kate', 'Peter', 'Bob', 'Alice')
('Peter', 'Alice', 'Bob', 'Kate')
('Peter', 'Alice', 'Kate', 'Bob')
('Peter', 'Bob', 'Alice', 'Kate')
('Peter', 'Bob', 'Kate', 'Alice')
('Peter', 'Kate', 'Alice', 'Bob')
('Peter', 'Kate', 'Bob', 'Alice')
```

Комбінації з двох імен

```
('Alice', 'Bob')
('Alice', 'Kate')
('Alice', 'Peter')
('Bob', 'Kate')
('Bob', 'Peter')
('Kate', 'Peter')
```